

# **ARTIFICIAL INTELLIGENCE ASSESSMENT – 1 REPORT**

**Course Code:** CSA1717 – Artificial Intelligence

**Submitted By**

**Name:** Krisha Meenatchi M

**Register Number:** 192511338

**Department:** Computer Science and Engineering

## **Acknowledgement**

I express my sincere gratitude to the Department of Computer Science and Engineering and my course faculty for providing me with the opportunity to complete this Artificial Intelligence Assessment successfully. This assessment enhanced my understanding of intelligent agents, search techniques, state-space problems, and graph search algorithms used in Artificial Intelligence. I also extend my heartfelt thanks to my friends and family members for their constant encouragement and support throughout this work. Finally, I thank everyone who directly and indirectly contributed to the successful completion of this assessment.

## **Table of Contents**

1. Introduction
2. Objective
3. Question 1 – Water Jug Problem
4. Question 2 – Mars Rover Intelligent Agent
5. Question 3 – Eight Queens Problem
6. Question 4 – OLA Cab Booking Agent
7. Question 5 – Uniform Cost Search
8. Python Implementation
9. Program Outputs
10. Conclusion
11. References

## **Introduction**

Artificial Intelligence (AI) is a branch of computer science that focuses on designing intelligent systems capable of performing tasks that normally require human intelligence. These tasks include reasoning, learning, decision making, planning, problem solving, perception, and understanding natural language. AI enables machines to analyze data, make informed decisions, and solve complex real-world problems efficiently.

Artificial Intelligence uses various search algorithms, intelligent agents, and optimization techniques to find suitable solutions for different applications. Intelligent agents perceive their environment using sensors, process the collected information, and perform appropriate actions through actuators to achieve predefined goals.

This assessment focuses on five important AI concepts including the Water Jug Problem, Mars Rover Intelligent Agent, Eight Queens Problem, OLA Cab Booking Agent, and Uniform Cost Search. These problems demonstrate the application of state-space search, intelligent agents, constraint satisfaction, goal-based reasoning, and graph search algorithms. Python programming language is used to implement these algorithms because of its simplicity, readability, and extensive support for Artificial Intelligence applications.

## **Objective**

The objectives of this assessment are:

- To understand the fundamental concepts of Artificial Intelligence.
- To study intelligent agents and their working principles.
- To understand state-space search problems.
- To formulate search problems using AI techniques.
- To implement Artificial Intelligence algorithms using Python.
- To understand graph search algorithms such as Uniform Cost Search.
- To analyze real-world applications of intelligent agents.
- To develop problem-solving skills using Artificial Intelligence techniques.

## QUESTION 1 – WATER JUG PROBLEM

### Problem Statement

The Water Jug Problem is one of the classical state-space search problems in Artificial Intelligence. Two unmarked jugs having capacities of 4 gallons and 3 gallons are provided. A water pump is available for filling the jugs, and water can also be poured from one jug to another or emptied onto the ground. The objective is to obtain exactly two gallons of water in the four-gallon jug using only the permitted operations. This problem demonstrates how Artificial Intelligence explores different states systematically until the desired goal state is reached.

### Problem Understanding

The Water Jug Problem is formulated as a state-space search problem where every state represents the quantity of water available in both jugs. A state is represented as:

**State = (x, y)**

where

- x represents the amount of water in the 4-gallon jug.
- y represents the amount of water in the 3-gallon jug.

The initial state is

**(0,0)**

where both jugs are empty.

The goal state is any state in which

**x = 2**

regardless of the amount of water in the second jug.

### Method Selection

The problem is solved using an **Uninformed Search** approach. The solution is obtained by repeatedly applying the valid operators such as filling, emptying, and pouring water between the two jugs until the required goal state is reached. Since there is no heuristic information available, every possible valid state is explored systematically.

## Solution Process

### Allowed Operators

- Fill the 4-gallon jug.
- Fill the 3-gallon jug.
- Empty the 4-gallon jug.
- Empty the 3-gallon jug.
- Pour water from the 4-gallon jug to the 3-gallon jug.
- Pour water from the 3-gallon jug to the 4-gallon jug.

### State Transitions

| Step | Action                   | 4-Gallon Jug | 3-Gallon Jug |
|------|--------------------------|--------------|--------------|
| 0    | Start                    | 0            | 0            |
| 1    | Fill 4-gallon jug        | 4            | 0            |
| 2    | Pour into 3-gallon jug 1 | 3            | 1            |
| 3    | Empty 3-gallon jug       | 3            | 0            |
| 4    | Pour remaining water     | 0            | 1            |
| 5    | Fill 4-gallon jug        | 4            | 1            |
| 6    | Pour into 3-gallon jug 2 | 2            | 3            |

### Interpretation of Result

After performing the above sequence of operations, the four-gallon jug contains exactly **2 gallons** of water. Thus, the goal state **(2,3)** is successfully reached using only the permitted operations of filling, emptying, and pouring water between the two jugs. The solution requires six valid state transitions to achieve the objective.

## QUESTION 2 – MARS ROVER INTELLIGENT AGENT

### Problem Statement

A Mars Rover is an autonomous intelligent agent designed to explore the surface of Mars without continuous human intervention. It collects information from its surroundings using sensors, analyzes rocks and soil, avoids obstacles, performs scientific experiments, and transmits valuable data back to Earth. Since the Martian environment is uncertain and continuously changing, the rover must make intelligent decisions based on the information it perceives. This problem focuses on identifying the rover's percepts, actions, operating environment, performance measures, and the most suitable AI agent architecture.

### Solution

The Mars Rover is considered a **Model-Based Utility-Based Intelligent Agent** because it maintains an internal model of its environment and selects actions that maximize mission success while minimizing risks such as energy consumption and obstacles. It continuously updates its knowledge based on sensor readings and chooses the best possible action to accomplish scientific objectives.

### Percepts

The rover gathers information from its surroundings through various sensors.

- Camera images and panoramic views of the Martian surface
- Soil and rock composition obtained using drills and spectrometers
- Position and orientation data
- Obstacle detection using proximity sensors
- Atmospheric conditions such as temperature, radiation, and dust levels
- Battery status and wheel movement information

### Operating Environment

The operating environment of the Mars Rover is characterized as follows:

- Partially Observable
- Stochastic

- Sequential
- Dynamic
- Continuous
- Single Agent

### **Actions**

The rover can perform the following actions:

- Move forward and backward
- Turn in different directions
- Capture images
- Drill rocks and collect soil samples
- Analyze collected samples
- Transmit scientific data to Earth
- Recharge using solar panels
- Re-plan its path when obstacles are detected

### **Performance Measures**

The efficiency of the Mars Rover is evaluated based on:

- Number of successful scientific observations
- Quality of collected samples
- Distance travelled safely
- Energy efficiency
- Successful avoidance of obstacles
- Communication reliability
- Mission completion within the specified time

## Agent Architecture

Environment



Sensors



Internal Model



Decision Making



Utility Evaluation



Actuators



Mars Exploration

## Result

The **Model-Based Utility-Based Agent** successfully performs autonomous exploration by continuously updating its internal knowledge, selecting optimal actions, avoiding hazards, collecting scientific samples, and transmitting valuable information to Earth.

## QUESTION 3 – EIGHT QUEENS PROBLEM

### Problem Statement

The Eight Queens Problem is a well-known Constraint Satisfaction Problem in Artificial Intelligence. The objective is to place eight queens on an 8×8 chessboard so that no queen attacks another queen. Since a queen can attack horizontally, vertically, and diagonally, the algorithm must determine a valid arrangement without conflicts. This problem demonstrates how search algorithms efficiently reduce the search space by eliminating invalid solutions during the search process.

## **Solution**

The Eight Queens Problem is solved using the **Incremental Search (Backtracking/Depth-First Search)** approach. Queens are placed one column at a time. Whenever a conflict occurs, the algorithm removes the previously placed queen and searches for another valid position until all eight queens are placed safely.

## **Problem Formulation**

### **Initial State**

Empty chessboard.

### **States**

Any arrangement of zero to eight queens where no previously placed queens attack each other.

### **Actions**

Place one queen in the leftmost empty column without violating any constraints.

### **Transition Model**

Returns a new board configuration after placing the queen.

### **Goal Test**

All eight queens are placed on the board without attacking each other.

### **Path Cost**

Not applicable because every placement has equal cost.

## **Architecture**

Start



Empty Board

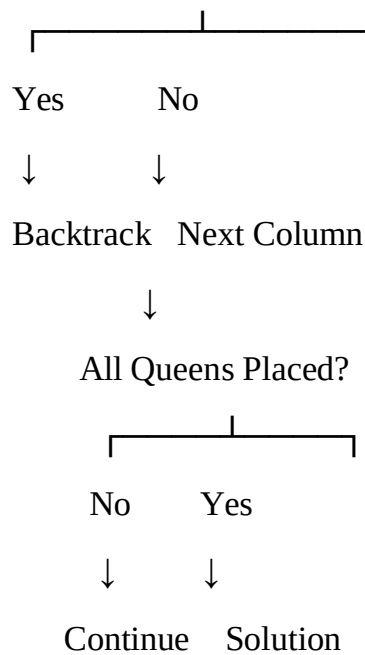


Place Queen



Conflict?





## Result

The incremental search approach successfully places all eight queens on the chessboard without any row, column, or diagonal conflicts. This formulation significantly reduces the search space and enables efficient problem solving.

## QUESTION 4 – OLA CAB BOOKING AGENT

### Problem Statement

The OLA Cab Booking System is an example of a Goal-Based Problem Solving Agent in Artificial Intelligence. A customer wishes to travel from one location to another by selecting a suitable cab based on comfort, fare, waiting time, and availability. The intelligent agent evaluates different cab options and selects the most appropriate one according to the user's requirements. This demonstrates how AI agents make rational decisions based on user goals.

### Solution

The OLA Cab Booking System functions as a **Goal-Based Agent**. The agent first formulates the user's goal, identifies all available cab options, compares them according to user preferences, selects the most suitable cab, and finally confirms the booking.

## **Pseudocode**

FUNCTION OLA\_CAB\_AGENT(Pickup, Destination, Preference)

Formulate Problem

Get Available Cabs

Filter Cabs Based on Preference

Search Best Route

Select Best Cab

Book Cab

Return Booking Details

END

## **Architecture**

User Request



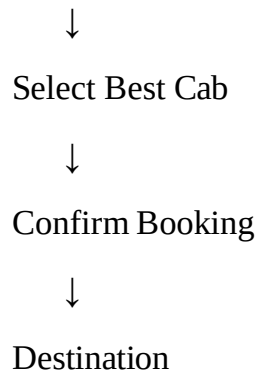
Goal Formulation



Available Cabs



Compare Options



### **Result**

The Goal-Based Agent successfully identifies the most suitable cab according to the user's comfort, cost, and availability preferences, resulting in an efficient and reliable booking process.

## **QUESTION 5 – UNIFORM COST SEARCH**

### **Problem Statement**

A logistics company operates a network of warehouses connected by routes having different travel costs. The objective is to determine the least-cost path from the starting warehouse to the destination warehouse while minimizing transportation expenses. Since every route has a different cost, the algorithm must always explore the path with the lowest cumulative cost before considering more expensive alternatives. This problem demonstrates the working of the Uniform Cost Search algorithm.

### **Solution**

Uniform Cost Search (UCS) expands the node having the minimum cumulative path cost first. A priority queue is maintained so that the node with the least path cost is always selected for expansion. UCS guarantees the optimal solution when all edge costs are non-negative.

### **Route Cost**

#### **Route Cost**

$S \rightarrow A \ 2$

## Route Cost

$S \rightarrow B$  5

$A \rightarrow B$  1

$A \rightarrow C$  8

$B \rightarrow D$  2

$D \rightarrow C$  1

$D \rightarrow G$  7

$C \rightarrow G$  3

## Architecture

Start



Priority Queue



Expand Lowest Cost Node



Generate Successors



Goal Test



Goal Reached

## Cost Comparison

| Path | Total Cost |
|------|------------|
|------|------------|

|                                                                           |          |
|---------------------------------------------------------------------------|----------|
| $S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow G$ | <b>9</b> |
|---------------------------------------------------------------------------|----------|

|                                                             |    |
|-------------------------------------------------------------|----|
| $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$ | 12 |
|-------------------------------------------------------------|----|

| Path                                                        | Total Cost |
|-------------------------------------------------------------|------------|
| $S \rightarrow A \rightarrow C \rightarrow G$               | 13         |
| $S \rightarrow B \rightarrow D \rightarrow C \rightarrow G$ | 11         |
| $S \rightarrow B \rightarrow D \rightarrow G$               | 14         |

## Result

Uniform Cost Search successfully identifies the least-cost path from warehouse **S** to warehouse **G**. The optimal path is  **$S \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow G$**  with a total cost of **9**, demonstrating that UCS always produces the optimal solution when all edge costs are positive.

## Conclusion

This assessment provided a comprehensive understanding of fundamental Artificial Intelligence concepts including intelligent agents, state-space search, constraint satisfaction problems, goal-based reasoning, and graph search algorithms. The Water Jug Problem demonstrated state-space search, the Mars Rover illustrated intelligent agent architecture, the Eight Queens Problem explained constraint satisfaction using incremental search, the OLA Cab Booking System represented goal-based decision making, and the Uniform Cost Search algorithm showed optimal path finding in weighted graphs. The implementation of these concepts using Python enhanced practical programming skills and demonstrated how Artificial Intelligence techniques can be applied to solve real-world problems effectively.